

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_c5395511-084\agent-a1b5a57f0b7245a77.jsonl`
- SHA-256: `4da8b58e23d624cf5920e9f75cd50f3b3fedd83f03e9ba9649ed4f3de3fa01de`
- Source modified: `2026-07-06T21:31:35+00:00`
- Imported at: `2026-07-08T16:00:09+00:00`
- project: `wf\_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:29:35.078Z)

Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there, or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (db._apply_schema on aiosqlite ":memory:").

WHAT F2 DOES:

- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video / _content_item_to_video / get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences (json.loads(policy.config_json)); TopicPreferenceStage (Tier 2): exclude-veto, include any/all, casefold substring match over source.title + content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty); _content_item_preview(...) builds a throwaway ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn, account, source, item) loads the default policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now: matched=engine.evaluate(); if matched and source: run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a /set (F1's merge) and are read via the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate: ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium (edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if handled / can't occur /

pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity.
FINDING (design_tests): Malformed topics config fails closed, dropping a filter-matched video with no user-visible recourse
severity low | src/tg_video_filter/policy.py:110
summary: TopicPreferenceStage.evaluate returns StageResult.fail on any json/ValueError from load_topic_preferences, so a persisted policy with a malformed topics object turns matched=True into matched=False and suppresses delivery.
scenario: If topics is ever persisted as e.g. {"topics": {"mode": "weird"}} or topics is a non-dict, every otherwise-matching video for that user is silently dropped (matched flipped to False, delivery skipped), and because there is no /set topics command a user cannot easily discover or repair the bad config. Confirmed via repro: config_json
'{"topics":{"mode":"weird"}}' yields passed=False reason='invalid topic config: topics.mode must be any or all'.
evidence: policy.py:110-112 catches (json.JSONDecodeError, ValueError) and returns StageResult.fail; on_new_message/backfill then set matched=topic_result.passed=False. Contrast db.load_filter_config which raises (loud) on malformed config; the stage instead fails silently-closed.
rec: Consider fail-open (treat invalid topic config as disabled, log a warning) so a malformed optional stage never suppresses an otherwise-matched delivery, or surface the error to the user. Document the fail-closed choice if intentional.

2. assistant (2026-07-06T21:29:36.967Z)

I'll verify this finding adversarially. Let me read the cited code first.

3. user (2026-07-06T21:29:37.816Z)

```
1      """Policy pipeline primitives for ADR 0012.
2
3      The policy layer is deliberately small and framework-free in F1: stages receive
4      an immutable context, return a structured result, and the evaluator stops on
5      the first failure. Later Track F phases add concrete stages.
6      """
7
8      from __future__ import annotations
9
10     import json
11     from dataclasses import dataclass, field
12     from typing import Protocol
13
14     from tg_video_filter.models import ContentItem, Policy, Source
15
16
17     @dataclass(frozen=True)
18     class PolicyContext:
19         """Immutable snapshot available to a policy stage."""
20
21         content_item: ContentItem
22         source: Source
23         policy: Policy
24         fetcher: object | None = None
25         budget: object | None = None
26
27
28     @dataclass(frozen=True)
29     class StageResult:
30         """Outcome of one policy stage."""
31
32         passed: bool
33         reason: str
34         stage_name: str
35         metadata: dict[str, object] = field(default_factory=dict)
36
37     @staticmethod
38     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
```

```

39         return StageResult(
40             passed=True,
41             reason=reason,
42             stage_name=stage_name,
43             metadata=metadata,
44         )
45
46     @staticmethod
47     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
48         return StageResult(
49             passed=False,
50             reason=reason,
51             stage_name=stage_name,
52             metadata=metadata,
53         )
54
55
56 class PolicyStage(Protocol):
57     """A single stateless gate in the policy pipeline."""
58
59     async def evaluate(self, ctx: PolicyContext) -> StageResult:
60         """Evaluate ``ctx`` and return a stage result."""
61         ...
62
63
64 @dataclass(frozen=True)
65 class TopicPreferences:
66     """Parsed ADR 0012 topic preference config."""
67
68     include: tuple[str, ...] = ()
69     exclude: tuple[str, ...] = ()
70     mode: str = "any"
71
72     @property
73     def enabled(self) -> bool:
74         return bool(self.include or self.exclude)
75
76
77 def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
78     """Parse ``topics`` from a raw policy config object."""
79     raw_topics = config.get("topics")
80     if raw_topics is None:
81         return TopicPreferences()
82     if not isinstance(raw_topics, dict):
83         raise ValueError("topics must be a JSON object")
84     mode = raw_topics.get("mode", "any")
85     if mode not in ("any", "all"):
86         raise ValueError("topics.mode must be 'any' or 'all'")
87     return TopicPreferences(
88         include=_string_tuple(raw_topics.get("include")),
89         exclude=_string_tuple(raw_topics.get("exclude")),
90         mode=str(mode),
91     )
92
93
94 def load_topic_preferences(policy: Policy) -> TopicPreferences:
95     """Load topic preferences from ``policy.config_json``."""
96     decoded = json.loads(policy.config_json)
97     if not isinstance(decoded, dict):
98         raise ValueError("policy config_json must decode to a JSON object")
99     return parse_topic_preferences(decoded)
100
101
102 def _string_tuple(value: object) -> tuple[str, ...]:
103     if value is None:

```

```

104         return ()
105     if not isinstance(value, list):
106         raise ValueError("topic include/exclude values must be arrays")
107     topics: list[str] = []
108     for item in value:
109         if not isinstance(item, str):
110             raise ValueError("topic include/exclude arrays must contain strings")
111         topic = item.strip()
112         if topic:
113             topics.append(topic)
114     return tuple(topics)
115
116
117 class TopicPreferenceStage:
118     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
119
120     stage_name = "topic-preferences"
121
122     async def evaluate(self, ctx: PolicyContext) -> StageResult:
123         try:
124             prefs = load_topic_preferences(ctx.policy)
125         except (json.JSONDecodeError, ValueError) as exc:
126             return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
127         if not prefs.enabled:
128             return StageResult.pass_(self.stage_name, "topic preferences disabled")
129
130         haystack = _topic_haystack(ctx)
131         matched_excludes = _matched_topics(prefs.exclude, haystack)
132         if matched_excludes:
133             return StageResult.fail(
134                 self.stage_name,
135                 f"excluded topic matched: {matched_excludes[0]}",
136                 matched_exclude_topics=list(matched_excludes),
137             )
138
139         if not prefs.include:
140             return StageResult.pass_(self.stage_name, "no include topics configured")
141
142         matched_includes = _matched_topics(prefs.include, haystack)
143         if prefs.mode == "any":
144             if matched_includes:
145                 return StageResult.pass_(
146                     self.stage_name,
147                     f"matched include topic: {matched_includes[0]}",
148                     matched_include_topics=list(matched_includes),
149                 )
150             return StageResult.fail(self.stage_name, "no include topics matched")
151
152         missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
153         if missing:
154             return StageResult.fail(
155                 self.stage_name,
156                 f"missing required topic: {missing[0]}",
157                 matched_include_topics=list(matched_includes),
158                 missing_include_topics=list(missing),
159             )
160         return StageResult.pass_(
161             self.stage_name,
162             "matched all include topics",
163             matched_include_topics=list(matched_includes),
164         )
165
166
167 def _topic_haystack(ctx: PolicyContext) -> str:

```

```

168         parts = [ctx.source.title or "", ctx.content_item.text or ""]
169         return "\n".join(parts).casefold()
170
171
172     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
173         return tuple(topic for topic in topics if topic.casefold() in haystack)
174
175
176     async def evaluate_pipeline(
177         ctx: PolicyContext,
178         stages: list[PolicyStage],
179     ) -> tuple[bool, list[StageResult]]:
180         """Evaluate stages in order and short-circuit on the first failure."""
181         results: list[StageResult] = []
182         for stage in stages:
183             result = await stage.evaluate(ctx)
184             results.append(result)
185             if not result.passed:
186                 return False, results
187         return True, results
188
189
190     __all__ = [
191         "PolicyContext",
192         "PolicyStage",
193         "StageResult",
194         "TopicPreferenceStage",
195         "TopicPreferences",
196         "evaluate_pipeline",
197         "load_topic_preferences",
198         "parse_topic_preferences",
199     ]
200

```

4. assistant (2026-07-06T21:29:42.173Z)

Now let me verify the claim by running a repro. I need to check the actual failure scenario and also verify how the caller wires `topic\_result.passed`.

5. user (2026-07-06T21:29:44.514Z)

```

272:         matched = result.matched
275:         topic_result = await _evaluate_topic_preferences(
289:         if topic_result is not None:
290:             reasons.append(topic_result.reason)
291:             matched = topic_result.passed
423:async def _evaluate_topic_preferences(
471:     "_evaluate_topic_preferences",

```

6. user (2026-07-06T21:29:47.049Z)

```

260             conn,
261             Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
262         )
263         if channel.id is not None:
264             source = await db.get_source_by_id(conn, channel.id)
265         except Exception:
266             log.warning(
267                 "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
268             )
269
270         text = _message_text(message)

```

```

271     result = engine_holder[ENGINE_KEY].evaluate(meta)
272     matched = result.matched
273     reasons = list(result.reasons)
274     if matched and source is not None:
275         topic_result = await _evaluate_topic_preferences(
276             conn,
277             user.id,
278             source,
279             _content_item_preview(
280                 user.id,
281                 source,
282                 message_id,
283                 document_id,
284                 meta,
285                 text,
286                 matched,
287             ),
288         )
289         if topic_result is not None:
290             reasons.append(topic_result.reason)
291             matched = topic_result.passed
292
293     record = VideoRecord(
294         user_id=user.id,
295         channel_id=channel_id,
296         message_id=message_id,
297         text=text,
298         document_id=document_id,
299         duration_s=meta.duration_s,
300         width=meta.width,
301         height=meta.height,
302         size_bytes=meta.size_bytes,
303         mime_type=meta.mime_type,
304         matched=matched,
305     )
306     try:
307         inserted = await db.insert_video(conn, record)
308     except sqlite3.IntegrityError:
309         # Lost a dedup race: a concurrent delivery of the same file
310         # (same document_id or same message instance) won the insert.
311         # The three-tier check above already classified this as new, so
312         # the collision means another task is processing (or has
313         # processed) it ? log and skip.
314         log.info(
315             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
316             user.id,
317             channel_id,
318             message_id,
319             document_id,

```

7. assistant (2026-07-06T21:29:49.286Z)

Now let me look at `\_evaluate\_topic\_preferences` to confirm it returns the StageResult (with `passed=False` on malformed config).

8. user (2026-07-06T21:29:49.991Z)

```

423     async def _evaluate_topic_preferences(
424         conn: aiosqlite.Connection,
425         account_id: int,
426         source: Source,
427         content_item: ContentItem,
428     ) -> StageResult | None:
429         """Evaluate ADR 0012 topic preferences for a candidate item."""
430         policy = await db.get_default_policy(conn, account_id)

```

```

431         if policy is None:
432             return None
433         return await TopicPreferenceStage().evaluate(
434             PolicyContext(content_item=content_item, source=source, policy=policy)
435         )
436
437
438     def _peer_id(peer: object) -> int:
439         """Best-effort numeric id for the peer of a message.
440
441         Telegram represents channel/group/user peers as different TL types; we
442         collapse them to a single int for dedup/storage. The exact subclass isn't
443         important ? only that the id is stable per chat.
444         """
445         # Each Peer* type has a field like channel_id / chat_id / user_id.
446         for field in ("channel_id", "chat_id", "user_id"):
447             val = getattr(peer, field, None)
448             if val is not None:
449                 return int(val)
450         # Fallback: hash the repr so we never collide with real ids (negative).
451         return -abs(hash(repr(peer)) % (2**31))
452
453
454     def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
455         """Best-effort channel title from the event, if available."""
456         chat = getattr(event, "chat", None)
457         return getattr(chat, "title", None) if chat is not None else None
458
459
460     class _NewMessageCallback:
461         """Structural type annotation for the handler returned by make*_handler."""
462
463         async def __call__(self, event: NewMessage.Event) -> None: ...
464
465
466     __all__ = [
467         "build_client",
468         "extract_video_meta",
469         "make_new_message_handler",
470         "_content_item_preview",

```

9. assistant (2026-07-06T21:29:53.732Z)

The wiring is confirmed. Now let me run a repro to verify the malformed-config-fails-closed behavior claimed.

10. user (2026-07-06T21:29:59.581Z)

Exit code 1

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\repro_f2.py", line 14, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~^~~~~~

```

```

File "C:\Users\avidu\AppData\Local\Temp\repro_f2.py", line 6, in main
    src = Source(id=1, user_id=1, channel_id=1, title="Sports News")
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 2 validation errors for Source
account_id
  Field required [type=missing, input_value={'id': 1, 'user_id': 1, '... 'title': 'Sports
News'}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
provider_source_id
  Field required [type=missing, input_value={'id': 1, 'user_id': 1, '... 'title': 'Sports
News'}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing

```

11. assistant (2026-07-06T21:30:02.300Z)

Let me check the actual model fields.

12. user (2026-07-06T21:30:04.031Z)

```

123:class Source(BaseModel):
149:class ContentItem(BaseModel):
186:class Policy(BaseModel):

```

13. user (2026-07-06T21:30:06.431Z)

```

123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)

```

```

160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
169     text: str | None = Field(default=None, description="Caption / post text (nullable in Phase 1)")
170     link_url: str | None = Field(default=None, description="For link/article posts (Phase 1: NULL)")
171     media_ref: str | None = Field(default=None, description="Provider media reference")
172     document_id: str | None = Field(
173         default=None,
174         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175     )
176     duration_s: int | None = None
177     width: int | None = None
178     height: int | None = None
179     size_bytes: int | None = None
180     mime_type: str | None = None
181     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per account")
182     matched: bool = False
183     seen_at: datetime = Field(default_factory=utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197

```

14. user (2026-07-06T21:30:16.209Z)

Exit code 1

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\repro_f2.py", line 23, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Temp\repro_f2.py", line 18, in main
    policy = Policy(config_json=cj) if hasattr(Policy, "model_fields") else None
    ~~~~~^~~~~~

```

```
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 2 validation errors for Policy
account_id
  Field required [type=missing, input_value={'config_json': '{"topics":{"mode":"weird"}}'},
input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
name
  Field required [type=missing, input_value={'config_json': '{"topics":{"mode":"weird"}}'},
input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
```

15. user (2026-07-06T21:30:18.801Z)

```
197
198     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199     account_id: int = Field(description="FK ? accounts.id")
200     name: str = Field(description="Policy name; 'default' for the account default
policy")
201     config_json: str = Field(
202         description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203     )
204     is_default: bool = Field(
205         default=False, description="Exactly one per account (UNIQUE partial index)"
206     )
207     created_at: datetime = Field(default_factory=_utcnow)
208     updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212     #
213     # These tables were created empty by Track B (_SCHEMA); Track C populates
214     # destinations (backfilled from filter_config_deprecated.config_json), writes
215     # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216     # defines the models for per-destination delivery state. The source/destination
```

16. user (2026-07-06T21:30:28.294Z)

```
'{"topics":{"mode":"weird"}}' -> passed= False | reason= invalid topic config: topics.mode must
be 'any' or 'all'
'{"topics":"nope"}' -> passed= False | reason= invalid topic config: topics must be a JSON
object
'{"topics":{"include":[1,2]}}' -> passed= False | reason= invalid topic config: topic
include/exclude arrays must contain strings
'not json' -> passed= False | reason= invalid topic config: Expecting value: line 1 column 1
(char 0)
'[]' -> passed= False | reason= invalid topic config: policy config_json must decode to a JSON
object
'{}' -> passed= True | reason= topic preferences disabled
'{"topics":{"include":["match"]}}' -> passed= True | reason= matched include topic: match
```

17. assistant (2026-07-06T21:30:34.437Z)

The behavior is confirmed: malformed topic config yields `passed=False`, and the caller at telethon_client.py:291 sets `matched = topic\_result.passed`, flipping a filter-matched video to `matched=False` and suppressing delivery.

Now the adversarial part: can this actually occur in practice? The severity hinges on whether a malformed `topics` config can be persisted. Let me check F1's `/set` merge path and whether the caption/config_json path can hold a valid FilterConfig blob but malformed topics. Let me check how config_json is written and whether `\_string\_tuple`/`parse` is applied on write.

18. user (2026-07-06T21:30:36.281Z)

```

87:     config_json TEXT NOT NULL
136:     config_json TEXT NOT NULL,
480:         config_json TEXT NOT NULL
483:         "user_id, config_json",
626:     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
628:     SELECT user_id, 'default', config_json, 1,
699:         CASE WHEN json_valid(config_json)
700:             THEN json_extract(config_json, '$.delivery_target') END,
705:         CASE WHEN json_valid(config_json)
706:             THEN json_extract(config_json, '$.delivery_target') END,
730:         CASE WHEN json_valid(fcd.config_json)
731:             THEN json_extract(fcd.config_json, '$.delivery_target') END,
889:     config_json=row["config_json"],
1257:     """INSERT INTO policies (account_id, name, config_json, is_default,
1261:     config_json=excluded.config_json,
1267:     policy.config_json,
1295: def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1298:     ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1304:     raise ValueError("policy config_json must decode to a JSON object")
1317:     as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1323:     return _policy_config_json_to_dict(policy.config_json)
1337:     config_json=_policy_config_dict_to_json(config),
1761:     Invalid ``config_json`` still raises from JSON/Pydantic validation; callers
1767:     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1773:     ADR 0012: future pipeline keys live in the same ``policies.config_json``
1782:     raw_config = _policy_config_json_to_dict(existing.config_json)

```

19. user (2026-07-06T21:30:38.659Z)

```

1770     async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1771         """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1772
1773         ADR 0012: future pipeline keys live in the same ``policies.config_json``
1774         object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1775         merge the flat config fields into the existing object so unknown policy
1776         keys survive legacy ``/set`` command updates.
1777         """
1778         existing = await get_default_policy(conn, user_id)
1779         raw_config: dict[str, object] = {}
1780         if existing is not None:
1781             try:
1782                 raw_config = _policy_config_json_to_dict(existing.config_json)
1783             except (json.JSONDecodeError, ValueError):
1784                 log.warning(
1785                     "resetting malformed policy config while saving FilterConfig user=%s "
1786                     "policy_id=%s",
1787                     user_id,
1788                     existing.id,
1789                     exc_info=True,
1790                 )
1791                 raw_config = {}
1792         raw_config.update(cfg.model_dump(mode="json"))
1793         await save_policy_config(conn, user_id, raw_config)
1794
1795
1796     # --- bulk helpers (used by the manager on startup) -----
1797
1798
1799     async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1800         conn: aiosqlite.Connection, user_id: int, limit: int = 100
1801     ) -> list[VideoRecord]:
1802         """Deprecated shim: list the account's content items projected onto VideoRecord."""
1803         conn.row_factory = aiosqlite.Row
1804         async with conn.execute(

```

```

1805         """SELECT ci.*, s.provider_source_id AS src_pid
1806         FROM content_items ci
1807         JOIN sources s ON s.id = ci.source_id
1808         WHERE ci.account_id = ?
1809         ORDER BY ci.id DESC LIMIT ?""",
1810         (user_id, limit),
1811     ) as cur:
1812         rows = await cur.fetchall()
1813     return [
1814         VideoRecord(
1815             id=r["id"],
1816             user_id=user_id,
1817             channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
1818             message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
1819             text=r["text"],
1820             document_id=(
1821                 _safe_int_cast(r["document_id"], "document_id")
1822                 if r["document_id"] is not None
1823                 else None
1824             ),
1825             duration_s=r["duration_s"],
1826             width=r["width"],
1827             height=r["height"],
1828             size_bytes=r["size_bytes"],
1829             mime_type=r["mime_type"],

```

20. assistant (2026-07-06T21:30:42.764Z)

Key observation: there is **no** `/set topics` command in this PR ? the finding states this, and F1's `save_filter_config` only merges flat `FilterConfig` fields (it never writes a `topics` key). So how would a malformed `topics` object get persisted?

Let me confirm there's no write path in this PR (or the codebase) that produces a `topics` object. The `topics` key is only ever *read*, never written by any code in this PR.

21. user (2026-07-06T21:30:46.317Z)

```

C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/db.py:1317:   as ``topics`` /
``security_checks`` / ``deep_inspect`` instead of validating
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:78:   """Parse ``topics`` from a
raw policy config object."""
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:79:   raw_topics =
config.get("topics")
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:80:   if raw_topics is None:
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:82:   if not
isinstance(raw_topics, dict):
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:83:       raise
ValueError("topics must be a JSON object")
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:84:   mode =
raw_topics.get("mode", "any")
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:86:       raise
ValueError("topics.mode must be 'any' or 'all'")
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:107:   topics: list[str] = []
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:113:   topics.append(topic)

```

22. assistant (2026-07-06T21:30:59.368Z)

Confirmed: no code path in this PR (or the codebase) writes a `topics` object into `config_json`. The `topics` key can only be present if an operator hand-edits the DB / config. So the trigger requires an out-of-band manual mutation.

Let me assess the verdict. The behavioral claim is **fully reproduced and correct**: a persisted malformed `topics` object silently flips `matched=True ? False` and suppresses delivery

(telethon_client.py:291). The finding itself already discloses that there is no ``/set topics`` command, so the recourse gap is real. This is a genuine design choice in this PR's diff (the ``except ? StageResult.fail`` at policy.py:125-126, wired at telethon_client.py:291).

The severity question: the finding is filed as **low**. It is fail-closed on optional-stage config that (a) cannot be produced by any code in this PR, only by manual DB editing, and (b) when it does occur, drops deliveries silently ? but the config that triggers it is itself invalid/operator-error. It's a latent design-inconsistency (contrast with ``load_filter_config`` raising loudly). Low is appropriate ? it's a real behavior in the diff but requires an out-of-band malformed config to manifest, and it's a hygiene/design-note about fail-open-vs-fail-closed rather than a bug on any reachable happy/error path the PR's own code produces.

Let me double-check one thing: does the malformed config **also** break the legacy ``load_filter_config`` / normal filtering path (making the whole account fail earlier, so topic stage never even runs)? That would refute the "silently drops otherwise-matched" claim.

23. user (2026-07-06T21:30:59.523Z)

```
1755     async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1756         """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1757
1758         ADR 0011 (Option 2A): ``filter_config`` is renamed to
1759         ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1760         instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1761         Invalid ``config_json`` still raises from JSON/Pydantic validation; callers
1762         do not currently treat malformed persisted config as recoverable.
1763         """
1764         policy = await get_default_policy(conn, user_id)
1765         if policy is None:
1766             return FilterConfig()
1767         return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1768
1769
```

24. assistant (2026-07-06T21:31:06.644Z)

This is the crucial refutation test. The trigger config is ``{"topics":{"mode":"weird"}}`` ? a config that has valid FilterConfig flat fields but a malformed ``topics`` sub-object.

``load_filter_config`` calls ``FilterConfig.model_validate(...)``. Whether it raises depends on whether FilterConfig is strict about extra fields. If ``FilterConfig`` forbids extra keys, then a ``topics`` key would make the **engine load** itself fail before topic evaluation, meaning the account can't process messages at all (a louder, different failure) ? the topic stage's fail-closed would never be the operative behavior. If FilterConfig ignores extras, the engine loads fine and the topic stage is the sole gate that flips matched. Let me test.

25. user (2026-07-06T21:31:11.116Z)

```
OK  {'topics': {'mode': 'weird'}} -> {'min_duration_s': 5, 'max_duration_s': 7200,
'target_height': 1080, 'max_size_mb': 2048, 'exclude_gif': True, 'exclude_round': True,
'exclude_nosound': False, 'delivery_mode': <DeliveryMode.FORWARD: 'forward'>, 'delivery_target':
'me'}
OK  {'delivery_target': '@x', 'topics': {'mode': 'weird'}} -> {'min_duration_s': 5,
'max_duration_s': 7200, 'target_height': 1080, 'max_size_mb': 2048, 'exclude_gif': True,
'exclude_round': True, 'exclude_nosound': False, 'delivery_mode': <DeliveryMode.FORWARD:
'forward'>, 'delivery_target': '@x'}
extra= None
```

26. assistant (2026-07-06T21:31:21.015Z)

This confirms the finding is **not refuted** by the engine-load path: ``FilterConfig`` ignores the extra ``topics`` key (extra defaults to ``ignore``), so ``load_filter_config`` succeeds, the engine runs normally and matches the video, and then the ``TopicPreferenceStage`` is the **sole** gate that fails-closed and flips ``matched=True ? False``. The described wrong behavior (silent suppression

of an otherwise-matched delivery) genuinely occurs.

Verdict analysis:

- **Behavior reproduced**: Yes, exactly as described.
- **Ships in this PR's diff**: Yes ? `except (json.JSONDecodeError, ValueError): return StageResult.fail(...)` at `policy.py:125-126`, wired to `matched = topic_result.passed` at `telethon_client.py:291`. Both are new in this PR.
- **Reachability caveat**: The malformed `topics` object cannot be produced by any code in this PR (no `/set topics` command; `save_filter_config` only merges flat fields). It requires manual DB/config editing. This is disclosed in the finding itself.
- **Severity**: Filed as **low**. It is a real, reproduced fail-closed design inconsistency in the diff (contrast: `load_filter_config` raises loudly on malformed config, line 1761-1762 even documents that), but requires an out-of-band malformed config and is a design-note about fail-open-vs-fail-closed. Low is correct.

The finding is CONFIRMED (the defect genuinely ships and produces the described behavior) with severity low. The "no user-visible recourse" framing is accurate given no `/set topics` command exists.

27. user (2026-07-06T21:31:35.644Z)

Structured output provided successfully